

Network Architecture and Programming, CS4254 Fall 2026

Project 1: Basic Network Programming

Due: 11:59:59 PM on September 29 (Tuesday), 2026

This project is worth 5% of your grade. You may only complete it in a group of two.

1 Description

This assignment is intended to familiarize you with writing simple network code. You will implement a client program which communicates with a server using sockets. The server will ask your program to solve hundreds of simple mathematical expressions. If your program successfully solves all of the expressions, then the server will return a secret flag that is unique for each student. If you receive the secret flag, then you know that your program has run successfully, and you will receive full credit for the assignment.

2 Language

You can write your code in whatever language you choose, as long as your code compiles and runs on unmodified rlogin Linux machines on the command line. Do not use libraries that are not installed by default on the rlogin Linux machines. Similarly, your code must compile and run on the command line. You may use IDEs during development, but do not turn in your IDE project without a Makefile. Make sure your code has no dependencies on your IDE.

3 Protocol

The server runs on the machines `sprinter2.cs.vt.edu` and `sprinter3.cs.vt.edu`, listening for requests on a TCP socket bound to port 27993. You can use either servers to test your client. This exercise has four types of messages: HELLO, STATUS, SOLUTION, and BYE. Each message is an ASCII string consisting of multiple fields separated by spaces (0x20) and terminated with a line feed (0x0a, \n). The maximum length of each message is 256 bytes. Messages are case sensitive.

The protocol works as follows. The client initiates the protocol by creating a TCP socket connection to the server. Once the socket is connected, the client sends a HELLO message to the server. The format of the HELLO message is:

```
cs4254fall2026 HELLO [your VT username]\n
```

In your program you should replace `[your VT username]` with your actual VT Username. You must supply your VT Username so the server can look up the appropriate secret flag for you. The server will reply with a STATUS message. The format of the STATUS message is:

```
cs4254fall2026 STATUS [a number] [a math operator] [another number]\n
```

The three variable fields represent a simple mathematical expression, e.g., "5 + 10". The server may return plus, minus, multiplication, or division expressions. All numbers will be between 1 and 1000. Your program must solve the mathematical expression and return the answer to the server in a SOLUTION message. The SOLUTION message has the following format:

```
cs4254fall2026 [the solution]\n
```

It is okay for the solution to be negative. In the case of division, round the answer down to the nearest integer (do not send floating point numbers to the server).

The server will respond to the SOLUTION message with either another STATUS message, or a BYE message. If the server terminates the connection, that means your solution was incorrect. If the server sends another STATUS message, your program must solve the expression and return another SOLUTION message. The server will ask your program to solve hundreds of expressions; the exact number of expressions is chosen at random. Eventually, the server will return a BYE message. The BYE message has the following format:

```
cs4254fall2026 [a 64 byte secret flag] BYE\n
```

Once your program has received the BYE message, it can close the connection to the server. If the server returns "Unknown_VT_Username" in the BYE message, that means it did not recognize the VT Username that you supplied in the HELLO message. Otherwise, the 64-byte string is your secret flag: write this value down, since you need to turn it in along with your code.

4 Your client program

Your client program must execute on the command line using the following standard POSIX syntax:

```
$ ./simpleclient [-p port] <hostname> <VT Username>
```

```
E.g. python simpleclient.py -p 27993 sprinter2.cs.vt.edu VTpid
```

```
E.g. python simpleclient.py sprinter3.cs.vt.edu VTpid
```

Your program must follow this command line syntax exactly, i.e., your program must be called `simpleclient` and it must accept these exact parameters. If you cannot name your program `simpleclient` (i.e., your program is in Java and you can only generate `client.class`) then you must include a script called `simpleclient` in your submission that accepts these parameters and then executes your actual program. Please confirm your program does conform exactly to the specification, or you may lose points.

The `-p port` parameter is optional; it specifies the TCP port that the server is listening on. If this parameter is not supplied on the command line, assume the port is 27993. The `<hostname>` parameter is required, and specifies the name of the server (either a DNS name or an IP address in dotted notation). The `<VT Username>` parameter is required.

You should manually record the secret flags that are given to all team members into a file `secret_flags`. You should include one flag per line, with one line for each team member. You will submit this file along with your project.

Your program should print exactly one line of output: the secret flag from the server's BYE message. If your program encounters an error, it may print an error message before terminating. Your program should not write any files to disk, including writing to the `secret_flags` file.

5 Other Considerations

5.1 Conform to protocol specifications and handle invalid messages

You may test your client code with our server as many times as you like. Your client should conform to the protocol described above, otherwise the server will terminate the connection silently. Please have your client code fully tested on the rlogin Linux machines before submission. **Note:** If your connection hangs during testing from an external network, we recommend testing directly from within the rlogin cluster to bypass any perimeter firewalls.

Your client program must verify the validity of messages by strictly checking their format, i.e., the server may send corrupted messages just to try and crash your software. If a received message is not as expected, such as an incorrect field or wrong message type, you must assert an error and terminate your program. You should be strict; if the returned message does not exactly conform to the specification above, you should assert an error. Remember that network-facing code should be written defensively.

6 Grading

This project is worth 5% of your grade. If your program compiles, includes a README file (see below), and you successfully submit the secret flags of all group members, then you will receive full credit. We will check all teams' code to make sure that it works correctly. All student code will be scanned by plagiarism detection software to ensure that students are not copying code from the Internet or each other.

7 Submitting your project

To turn in your project, you should submit your (thoroughly documented) code along with three other files:

- A plain-text (no Word or PDF) **README** file. In this file, you should describe your high-level approach, the challenges you faced, a list of properties/features of your design that you think is good, and an overview of how you tested your code.
- A file called **secret_flags**. This file should contain the secret flags of all group members, one flag per line, in plain ASCII.
- Any **Makefile** that compiles your code.

For the final submission, you should group your files into a compressed file (*.tar.gz*) and submit through Canvas. Please name your compressed file as *group#_project1.tar.gz* (e.g., *group1_project1.tar.gz*). Only one group member needs to submit your project. You must submit your project by **11:59:59 PM on September 29, 2026**.

Useful advice:

- Get started early and come to the instructor's office hours.
- Use Canvas to post project-specific questions.